

PKCERT AI & Software Development Internship

Task 16: End-to-End Machine Learning Pipeline

Project

Abdullah Amir

AI and Software Development
23 July 2026

This report covers the theory and the results. The full code, outputs and plots live in the notebook `house_price_pipeline.ipynb`. The dataset is **Ames Housing** (De Cock, 2011): 1,460 residential sales in Ames, Iowa, 79 raw features, not used in any earlier task. This is the capstone task, and it is treated as one: a single pipeline carries the data from raw CSV to a documented, evaluated model, applying preprocessing, feature engineering and model comparison together rather than as separate exercises.

Part A: Dataset Selection & Data Preparation

What the features are. Each row is one house sale: a mix of 37 numeric measurements (areas, room counts, year built, etc.) and 43 categorical quality/type ratings (`OverallQual`, `KitchenQual`, `Neighborhood`, and so on). The target, `SalePrice`, is right-skewed (skew 1.88), so the model is trained on $\log(1 + \text{SalePrice})$, which is close to symmetric (skew 0.12); every error metric reported below is converted back to dollars for interpretability.

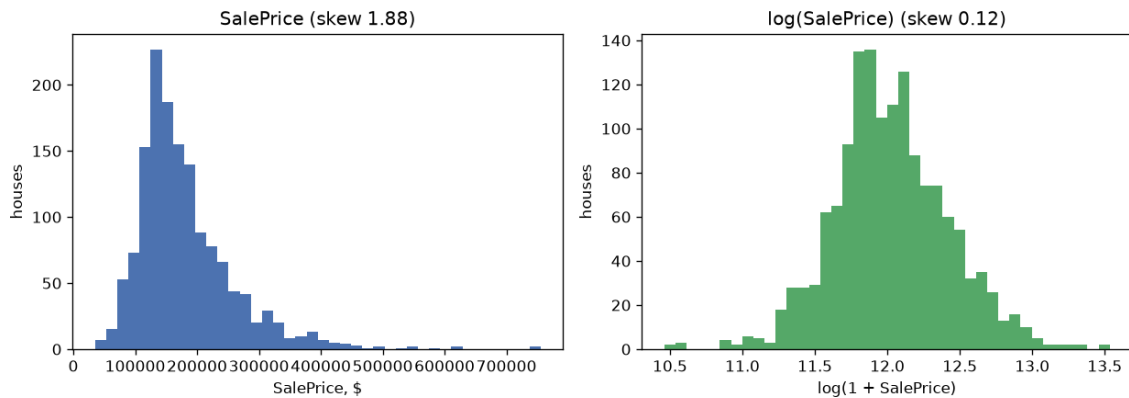


Figure 1: SalePrice before and after the log transform.

Outliers, checked individually rather than filtered by one threshold. The dataset's own author documents two specific outliers. The obvious shortcut is to drop every house above some `GrLivArea` cutoff, but two of the four largest houses in this data are simply large, expensive houses, not outliers – dropping them would throw away real signal for no reason.

Index	GrLivArea	SalePrice	Verdict
1298	5,642	\$160,000	dropped – outlier
523	4,676	\$184,750	dropped – outlier
1182	4,476	\$745,000	kept – genuinely large and expensive
691	4,316	\$755,000	kept – genuinely large and expensive

Table 1: Only 2 of the 4 largest houses are genuine outliers.

Missing values: most of them do not mean “missing”. In this dataset’s data dictionary, NaN in a quality/type column almost always means the feature simply does not exist on that house – PoolQC = NaN means *no pool*, not *unknown pool quality*. Treating every NaN as something to statistically impute would be a real mistake; those columns are filled with an explicit "None" / 0 instead, and true imputation is reserved for the columns that are genuinely incomplete: LotFrontage (259 rows, filled with the *neighbourhood* median, since street frontage tracks a neighbourhood’s typical lot shape far better than one dataset-wide number) and Electrical (1 row, mode).

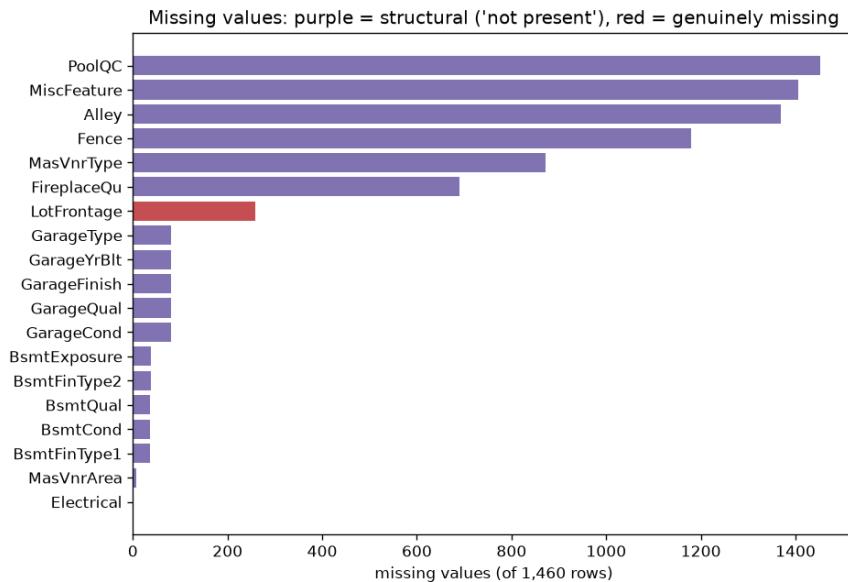


Figure 2: 19 columns have missing values; all but one are structural, not genuine gaps.

Feature engineering. Eight features built from domain knowledge of how houses are actually priced: TotalSF (basement + 1st + 2nd floor area), HouseAge and RemodAge (years since built / remodelled, relative to sale year), TotalBath (a single weighted bathroom count), and four presence flags (HasPool, HasGarage, HasFireplace, Has2ndFloor).

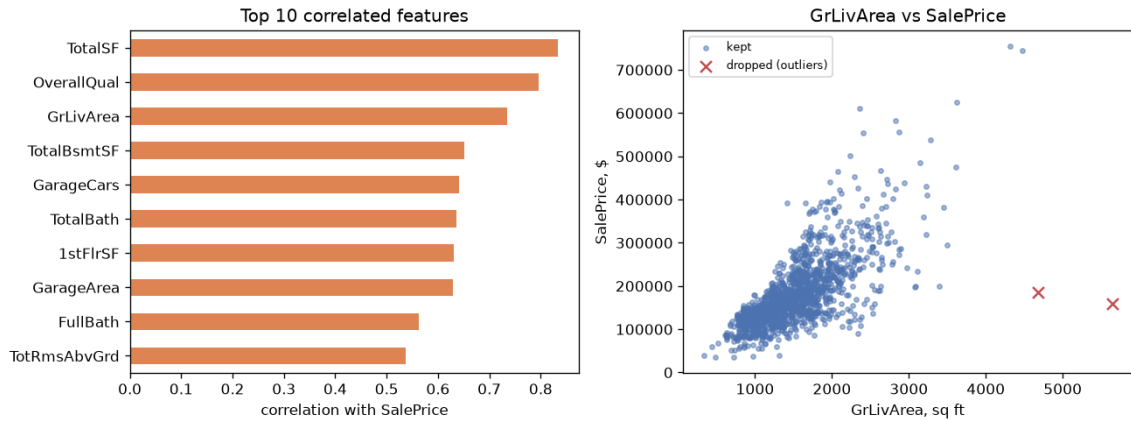


Figure 3: Left: top 10 features correlated with SalePrice – the engineered TotalSF leads. Right: the outlier decision, visualised.

The engineered feature is measurably better, not just convenient. TotalSF correlates with SalePrice at **0.833**, higher than either raw component it was built from (1stFlrSF 0.632, TotalBsmntSF 0.651) and higher than GrLivArea alone (0.735) – a checked reason to keep it, not an assumption that combining features must help.

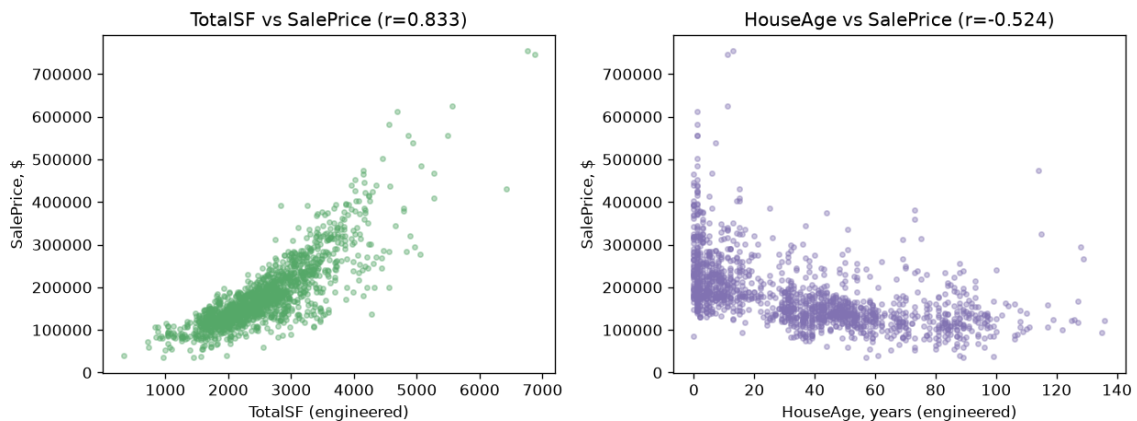


Figure 4: The two strongest engineered features against SalePrice.

Part B: Model Development

A single `ColumnTransformer` handles both feature types: numeric columns are median-imputed (for any residual gaps) and standard-scaled; categorical columns are most-frequent-imputed and one-hot encoded, expanding to 87 total features. Two models share this exact preprocessing inside one `Pipeline` object each, so the comparison below isolates the *model*, not the pipeline around it.

Model	CV RMSE (log)	CV R ²	Test R ²	Test RMSE	Test MAE
Ridge Regression	0.1120 ± 0.010	0.9191	0.9095	\$20,155	\$14,611
Random Forest	0.1353 ± 0.013	0.8829	0.8764	\$23,600	\$16,367

Table 2: 5-fold cross-validation on the training set, then held-out test metrics.

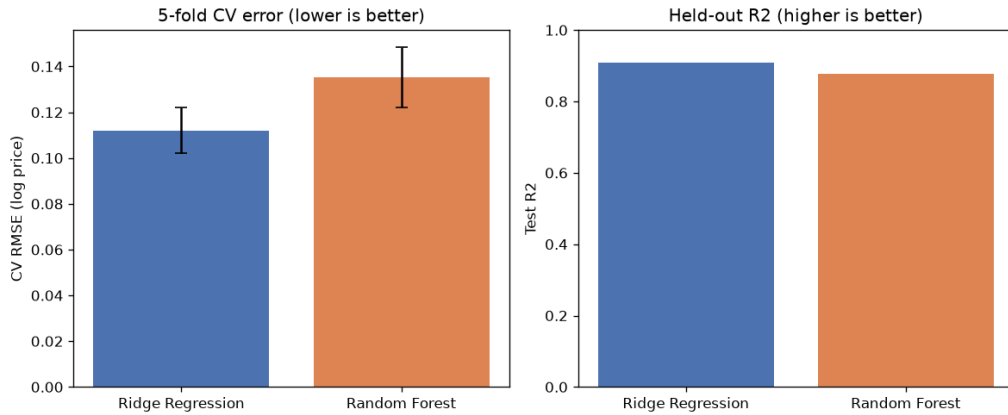


Figure 5: Ridge ahead of Random Forest in both cross-validation and on the test set.

Ridge beat Random Forest on every metric, in cross-validation and on the held-out test set alike. With 87 features after one-hot encoding and 1,166 training rows, most of the real signal – `OverallQual`, `TotalSF`, and dozens of one-hot category indicators – genuinely does relate to log-price close to linearly, which plays to Ridge’s strength, and its regularisation keeps the wide, sparse categorical block from overfitting. The Random Forest used here was not hyperparameter-tuned (400 trees, default depth); Part C returns to this as a stated limitation rather than a general verdict on tree ensembles.

Part C: Model Evaluation

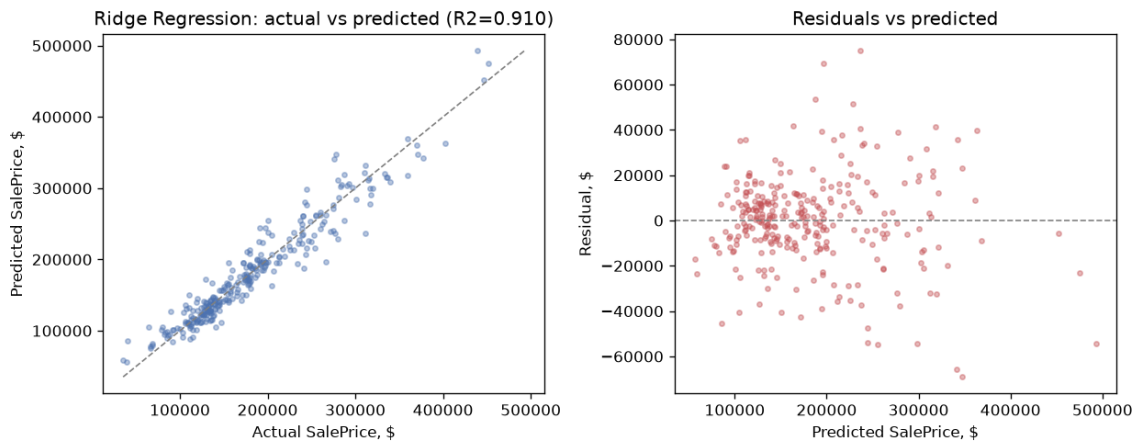


Figure 6: Ridge Regression, the chosen model: actual vs predicted, and residuals.

Reading the diagnostics. The actual-vs-predicted scatter tracks the diagonal closely across the bulk of the price range (\$100k–\$300k, where most of the data sits). The residual plot shows two things worth stating plainly rather than hiding behind the single R^2 number: residuals are roughly centred on zero across the price range (no systematic over- or under-prediction), but their *spread* widens for more expensive houses – a handful of \$300k+ homes have \$50k+ errors, while cheaper homes cluster much tighter. This is heteroscedasticity, and it means the model is more trustworthy for a typical Ames home than for the small number of high-end properties, a fair limitation to disclose rather than average away.

Strengths. Ridge Regression on this feature set reaches $R^2 = 0.910$ and a mean absolute

error of about \$14,600 – roughly 9% of a typical sale price – using a fully interpretable linear model where every coefficient has a direct, inspectable meaning. Training on log-price and converting back to dollars for every reported metric keeps the numbers both statistically sound and immediately usable by a non-technical reader.

Limitations. The 1,458-house sample is specific to Ames, Iowa in the years it was collected; nothing here should be expected to transfer to another city’s housing market without retraining. The Random Forest was not hyperparameter-tuned (Day 10 of this internship covered that skill separately, and applying it here was out of scope for this task), so “Ridge beat Random Forest” should be read as “beat this particular untuned Random Forest,” not as a general claim that linear models beat tree ensembles on housing data. Error variance grows with price, so predictions for the most expensive homes in the dataset carry the least confidence.

Part D: Project Documentation

The comprehensive project write-up – dataset description, methodology, implementation details, results and conclusion – lives in `README.md`, and the full, commented source is `house_price_pipeline.py` together with this notebook. Both are structured to mirror the four parts of this task (A: data preparation, B: model development, C: evaluation, D: this documentation) so a reader can trace every claim back to the exact code and figure that produced it.

Conclusion. The thread running through this capstone is that every step earns its place by being checked against the data, not assumed from habit: the log transform because the skew numbers say so, the two dropped rows because they were individually verified as outliers rather than swept up by a blanket threshold, the “None”/0 fill because the data dictionary says most of those gaps are not gaps at all, and `TotalSF` because it measurably outperforms the features it was built from. The result is a model that is both accurate ($R^2 = 0.91$, $MAE \approx 9\%$ of price) and honest about where it is weaker – expensive, atypical homes – rather than a single headline number standing in for the whole project.

The notebook and README are on GitHub: <https://github.com/AbdullahAmir06/ai-internship-abdullahamir>